

Continue Statement in C# with Examples

In this article, I am going to discuss **Continue Statement in C#** Language with Examples. Please read our previous articles, where we discussed **Break Statement in C#** with Examples. At the end of this article, you will understand what is Continue Statement and when and how to use the continue statement in C# language with examples.

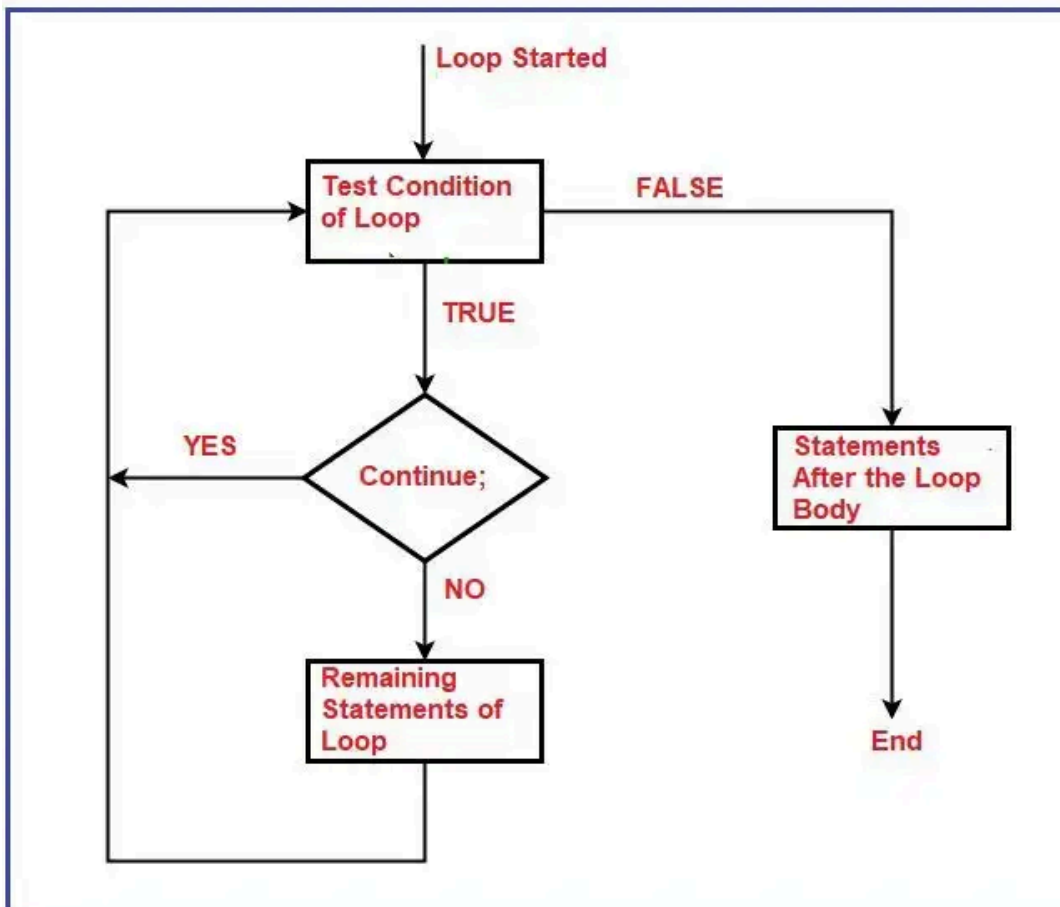
Continue Statement in C# Language:

In C#, continue is a keyword. By using the continue keyword, we can skip the statement execution from the loop body. Like the **break statement**, the use of the continue statement is also optional but if you want to use then you can use it only within the loop body.

If we know the maximum number of repetitions but we have some condition and when the condition is satisfied, we need to skip the statement execution from the loop body and we need to continue the loop execution for the next iteration, and this is possible in C# by using the continue statement in C#. The Continue Statement in C# provides a convenient way to immediately start the next iteration of the enclosing FOR, WHILE, Do While, and for each loop.

The **BREAK** statement terminates the loop, whereas the **CONTINUE** statement skips only the current loop iteration, and allows the next loop iteration to proceed. The continue statement is almost always used with the if...else statement.

Continue Statement Flowchart:

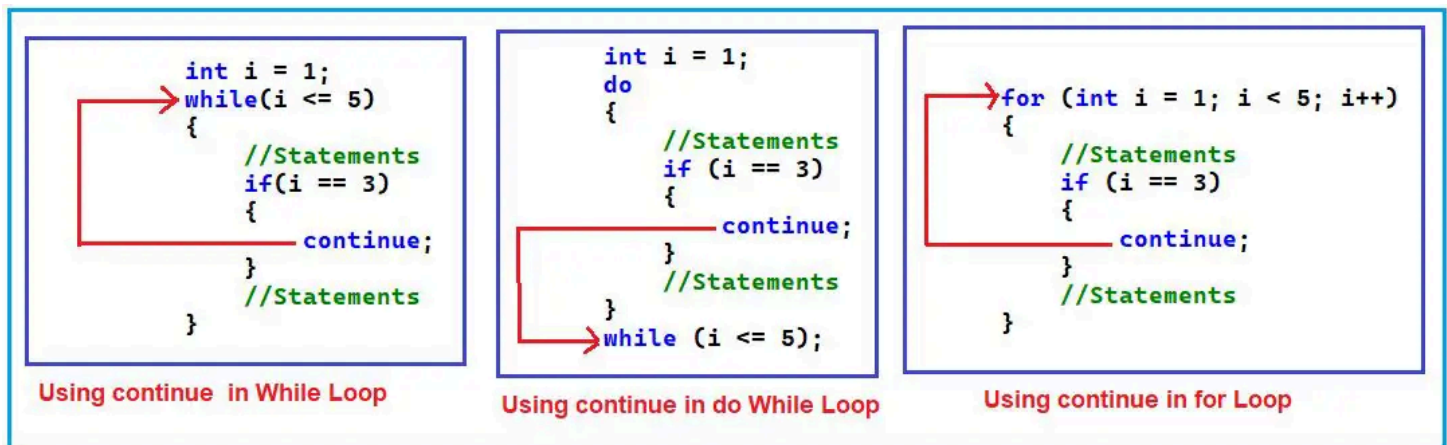


The continue statement in C# is very similar to the break statement, except that instead of terminating the loop, it will skip the current iteration and continue with the next iteration. That means the continue statement skips the rest of the body of the loop and immediately checks the loop's condition. If the loop's condition remains true, the loop's execution continues.

Syntax: `continue;`

How Does the Continue Statement Work in C# Language?

Now, let us understand how to use the continue statement inside the loop and see how the continue statement exactly works in C#. To understand this, please have a look at the following image. Here, I am showing how to use the continue statement inside do while, while, and for loop and how exactly the continue statement works.



If you notice the above code, we have written the if conditional statement inside the loop body, and within the if condition block, we have written the continue statement. So, when the loop executes, in each iteration, the if condition will be checked and if the condition is false, then it will execute the statements which are present after the if block and continue with the next iteration. Now, what happens when the if condition is true? Once the if condition is evaluated to true, then the if block will be executed, and once the continue statement within the if block is executed, it will skip the execution of the statements which are present after the continue statement and continue with the execution of the next iteration of the loop.

Example to Understand Continue Statement in C# Language:

In the below example, we have provided the condition for the loop to be executed 5 times i.e. starting from 1 value to 5. But our requirement is when the 1 value becomes 3, we need to skip the loop body execution and continue with the next iteration. In this case, we need to write if condition inside the loop body and check whether the current 1 value is equal to 3 or not. If it is not equal to 3, then continue the execution of the for loop and execute the next iteration. But if the 1 value is 3, then, the if condition will return true, and, in that case, the continue statement will be executed and once the continue statement is executed, it will not execute the rest of the statement of the current iteration and will move the control to continue with the next iteration.

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    for (int i = 1; i <= 5; i++)
    {
        if (i == 3)
        {
            continue;
        }
        Console.WriteLine($"I : {i}");
    }

    Console.ReadKey();
}
}

```

Output:

```

I : 1
I : 2
I : 4
I : 5

```

Note: When the continue statement is executed within the loop body then control will pass back to the condition without executing the remaining statement.

Using Continue Statement inside the nested loop:

The continue statement starts a new iteration of the closest enclosing iteration statement (for, for each, while, or do loop). For a better understanding, please have a look at the below example. Here, the outer loop will execute 5 times. Again, for each outer loop iteration, we have written the inner for loop condition to execute 5 times. But, inside the inner loop body, we have written the continue statement using the if condition, and when the inner value is equal to 3, it will skip the remaining statement execution.

```

using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int outer = 1; outer <= 5; outer++)
            {
                Console.WriteLine($"Outer: {outer}");
                for (int inner = 1; inner <= 5; inner++)
                {
                    if (inner == 3)
                    {
                        continue;
                    }
                }
            }
        }
    }
}

```

```

        }
        Console.WriteLine($"    Inner: {inner}");
    }
    Console.WriteLine();
}
Console.ReadKey();
}
}
}

```

Output:

```

Outer: 1
    Inner: 1  Inner: 2  Inner: 4  Inner: 5
Outer: 2
    Inner: 1  Inner: 2  Inner: 4  Inner: 5
Outer: 3
    Inner: 1  Inner: 2  Inner: 4  Inner: 5
Outer: 4
    Inner: 1  Inner: 2  Inner: 4  Inner: 5
Outer: 5
    Inner: 1  Inner: 2  Inner: 4  Inner: 5

```

We should avoid using “Continue Statement” where it’s possible. The continue statement executes some statements of the loop and then exits the loop without executing some statements after it. We can use the if statement for this purpose instead of continue. In the below example, we are not using the continue statement, using the if condition only we are achieving the same output.

```

using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int outer = 1; outer <= 5; outer++)
            {
                Console.WriteLine($"Outer: {outer}");
                for (int inner = 1; inner <= 5; inner++)
                {
                    if (inner != 3)
                    {
                        Console.WriteLine($"    Inner: {inner}");
                    }
                }
                Console.WriteLine();
            }
            Console.ReadKey();
        }
    }
}

```

```
}  
}
```

Output:

```
Outer: 1  
  Inner: 1  Inner: 2  Inner: 4  Inner: 5  
Outer: 2  
  Inner: 1  Inner: 2  Inner: 4  Inner: 5  
Outer: 3  
  Inner: 1  Inner: 2  Inner: 4  Inner: 5  
Outer: 4  
  Inner: 1  Inner: 2  Inner: 4  Inner: 5  
Outer: 5  
  Inner: 1  Inner: 2  Inner: 4  Inner: 5
```

Some tricky Questions Related to C# Continue Statement.

Question1: What will be the output in the below program?

```
using System;  
namespace JumpStatementDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            int a = 10;  
            while (a <= 25)  
            {  
                a += 2;  
                if (a > 15 && a < 20)  
                    continue;  
                Console.Write($"{a} ");  
            }  
            Console.ReadKey();  
        }  
    }  
}
```

Output: 12 14 20 22 24 26

Question2: What will be the output in the below program?

```
using System;  
namespace JumpStatementDemo  
{
```

```

class Program
{
    static void Main(string[] args)
    {
        int a = 1;
        while (a <= 50)
        {
            Console.Write($"{a} ");
            if (a >= 5 && a <= 35)
                continue;
            a = a + 2;
        }
        Console.ReadKey();
    }
}

```

Output: **infinite loop**

Question3: What will be the output in the below program?

```

namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 8;
            while (a <= 42) ;
            {
                a += 2;
                if (a >= 20 && a <= 30)
                    continue;
                Console.Write($"{a} ");
            }
            Console.ReadKey();
        }
    }
}

```

Output: **Error CS0139 No enclosing loop out of which to break or continue**

Note: When the semicolon (;) is placed after the while then it becomes a dummy loop. When the dummy loop is created then the compiler will create an empty body without any statement and the current body becomes outside. Then automatically continue is placing outside then it becomes an error.

In the next article, I am going to discuss **Goto Statement in C#** Language with Examples. Here, in this article, I try to explain **Continue Statement in C# Language** with Examples. I hope you enjoy this Continue Statement in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.